

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

**Лабораторная работа №1
по курсу “Компьютерная графика”**

Основы 3D графики

Выполнил: Бахшалиев Максим Анарович

Группа: М8О-311БВ-24

Преподаватель: В. Д. Бахарев

Москва, 2026

Условие

Целью лабораторной работы является знакомство с основами 3D-графики: построением простых трёхмерных объектов, их проекцией на 2D-плоскость, а также работой с матрицами перспективной и ортографической проекции и аффинными преобразованиями объектов в реальном времени.

Для выполнения работы используется графический API Vulkan, библиотека GLFW для создания окна и обработки ввода, а также библиотека Dear ImGui для построения интерфейса управления сценой.

Индивидуальный вариант задания — №4: «Усечённый правильный тетраэдр» — многогранник с 12 вершинами, 4 шестиугольными и 4 треугольными гранями, каждая из которых является правильным многоугольником.

Помимо базового варианта задания реализованы следующие дополнительные пункты:

1. Переключение матрицы проекции (ортографическая/перспективная) через интерфейс.
2. Элементы интерфейса для изменения позиции, поворота и масштаба фигуры.
3. Анимация движения фигуры по круговой траектории с возможностью паузы/воспроизведения, изменения скорости анимации и радиуса траектории.
4. Изменение цвета фигуры через элемент интерфейса ColorEdit.
5. Процедурное назначение цвета каждой вершине в зависимости от её положения в локальной системе координат фигуры, с последующим умножением результата на цвет, выбранный пользователем в интерфейсе.

Метод решения

Координаты вершин усечённого тетраэдра получены геометрическим построением: взят правильный тетраэдр с вершинами $(1,1,1)$, $(1,-1,-1)$, $(-1,1,-1)$, $(-1,-1,1)$, и каждое его ребро усечено на $1/3$ длины с обоих концов. В результате получено 12 новых вершин; корректность построения (4 шестиугольные и 4 треугольные грани)

проверена построением выпуклой оболочки точек. Полученный многогранник триангулирован в 20 треугольников для отрисовки граней, и отдельно выделен список из 18 рёбер для отрисовки контура.

Отображение сцены построено на классическом графическом конвейере Vulkan. Создаются базовые объекты `VkInstance`, `VkPhysicalDevice` и `VkDevice`, цепочка показа (`VkSwapchainKHR`), `render pass` с цветовым вложением и вложением глубины, а также графический `pipeline` (`VkPipeline`) с вершинным и фрагментным шейдерами, настроенными стадиями растеризации (с отключённым `back-face culling`), проверкой глубины (`depth test`) и динамическими состояниями `viewport/scissor`.

Геометрия фигуры передаётся в вершинный буфер (`VkBuffer` с позициями вершин) и два индексных буфера: один — список индексов для отрисовки треугольников граней (`VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST`), второй — список индексов рёбер для отдельного прохода отрисовки линий (`VK_PRIMITIVE_TOPOLOGY_LINE_LIST`). Контур рисуется вторым, отдельным `pipeline` поверх уже закрашенной фигуры с небольшим отрицательным `depth bias`, чтобы избежать `z-fighting` с гранями.

Общие для всей сцены данные — матрицы модели, вида и проекции, а также выбранный пользователем цвет — хранятся в `uniform-буфере`, который обновляется каждый кадр и связывается с шейдером через систему дескрипторов (`VkDescriptorSetLayout`, `VkDescriptorPool`, `VkDescriptorSet`).

Матрица модели строится как произведение матриц сдвига, поворота вокруг трёх осей и масштабирования — аффинное преобразование, выраженное через однородные координаты. Матрица вида задаётся функцией `glm::lookAt` с фиксированным положением камеры. Матрица проекции — перспективная (`glm::perspective`) или ортографическая (`glm::ortho`) — выбирается пользователем в интерфейсе `ImGui` и применяется в вершинном шейдере вместе с аппаратно выполняемым перспективным делением.

Анимация движения по траектории реализована на CPU: на каждом кадре накапливается время (с учётом состояния паузы и скорости), и на его

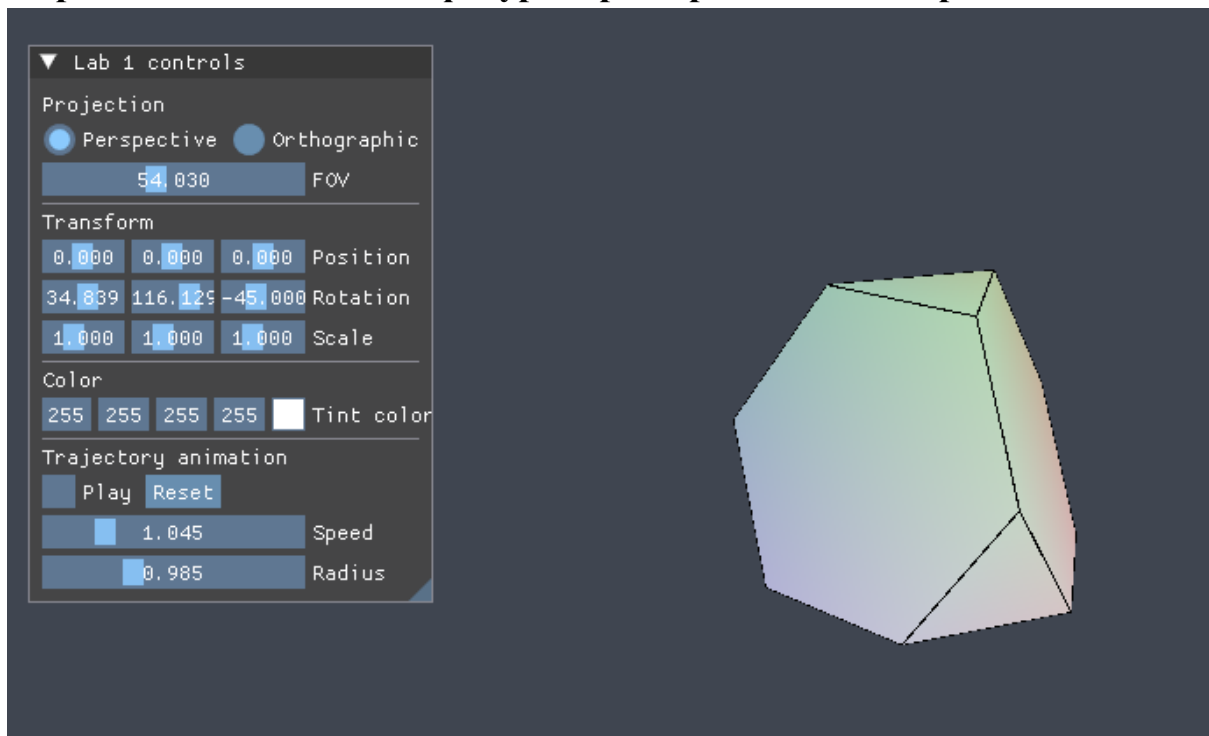
основе вычисляется смещение фигуры по кругу (через $\cos/\sin$ от накопленного времени и радиуса) и дополнительный поворот вокруг вертикальной оси.

Процедурный цвет вершины вычисляется непосредственно во вершинном шейдере на основе её локальных координат и умножается на цвет, выбранный пользователем (значение из `uniform`-буфера). Результирующий цвет интерполируется по поверхности треугольника аппаратно, по барицентрическим координатам, и передаётся во фрагментный шейдер.

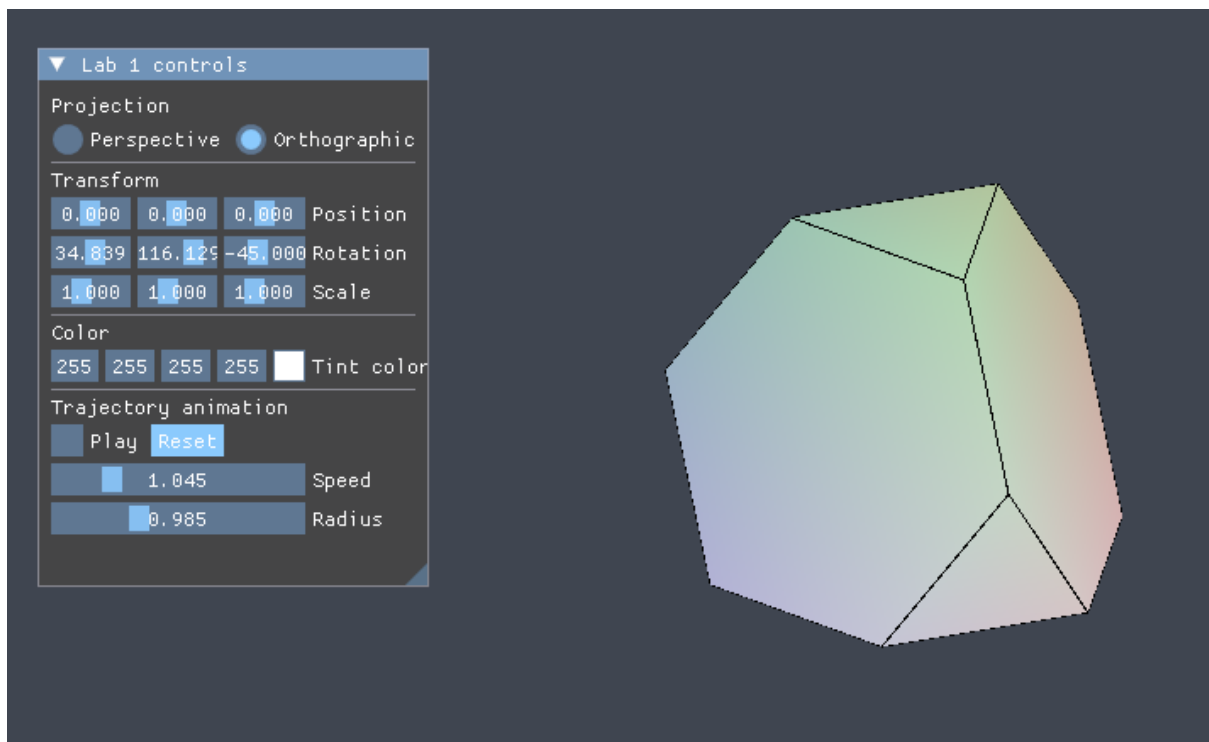
Результаты

Ниже приведены скриншоты работающего приложения, демонстрирующие выполнение базового задания и реализованных дополнительных пунктов.

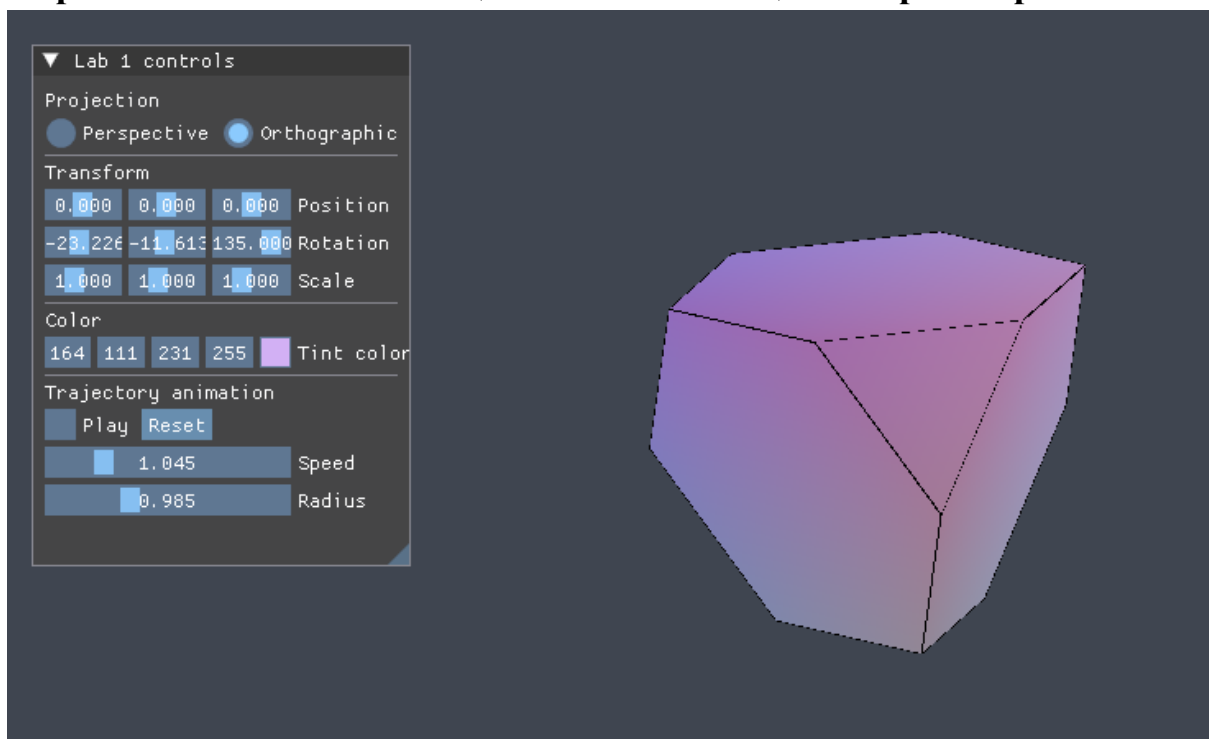
Скриншот 1. Общий вид фигуры при перспективной проекции.



Скриншот 2. Фигура при ортогографической проекции.



Скриншот 3. Изменённый цвет и/или анимация по траектории.



Выводы

Я научился работать с Vulkan API: разбираться в графическом конвейере (шейдеры, растеризация, depth test), строить 3D-объекты и выводить их на экран через вершинные и индексные буферы. Понял, как

через однородные координаты и аффинные преобразования задаются позиция, поворот и масштаб объекта, и в чём разница между перспективной и ортографической проекцией. Разобрался, как передавать данные в шейдер через uniform-буферы и дескрипторы, и как работает интерполяция цвета по поверхности треугольника. Также столкнулся с ошибкой в координатах вершин усечённого тетраэдра — фигура рисовалась неправильно, пришлось пересчитать координаты и проверить их через построение выпуклой оболочки, после чего всё заработало корректно.